

DetailWeave 4K：二つの分割候補から細部を選ぶ高精細拡大

DetailWeave 4K: Input-Preserving Local Selection of Two Tiled Candidates

あいきみ

要旨：大画像処理では、画像と潜在画像の変換を小領域へ分ける分割VAEと、一つの格子で画像から画像への局所生成を行う分割拡大が広く使われる。前者は変換の省メモリ化であり、新しい細部を生成しない。後者はGPU作業領域を固定できる一方、格子位置によって細線が変わる。本稿は、半格子ずらした二つの完成候補からA、B、入力維持を局所選択するDetailWeave 4Kを示す。Lanczos補間、単一格子分割、提案法を2897×4096で比較した。本手法は特定の画像モデルへ依存しない。

キーワード：4Kアップスケール、分割VAE、分割生成、局所選択、入力維持

1. はじめに

画像を大きくする最も単純な方法は、周囲の画素から新しい画素を計算する補間である。Lanczos補間は構図と色を保つが、元画像にない髪の毛の細線や布目は生成しない。生成モデルを使えば細部を描き足せる一方、大画像を一度に扱うには多くのメモリが必要になる。

現在はVAEの変換を重なり付き領域へ分ける分割VAEが一般化している。しかし、これは入口と出口の変換を省メモリ化する機構であり、拡散生成自体を分けない。局所生成まで分ける単一格子法は大画像を扱えるが、領域位置が変わると同じ毛束や文字状の線が数ピクセルずれる。

2. 比較する三つの拡大法

2.1 Lanczos補間

入力画素だけから拡大する。生成による形の変化はないが、新しい意味の細部も増えない。本稿では入力由来の形を確認する基準とする。

2.2 分割VAEと単一格子分割

潜在拡散モデルではVAEが画像と潜在画像を相互変換する[1]。分割VAEはこの符号化と復号だけを重なり付き領域へ分け、境界を重み付き平均する。指示文による生成や細部選択は行わない。実測比較には、画像から画像への局所生成と候補整形を共有し、一つの格子だけを採用する単一格子版を用いる。

2.3 DetailWeave 4K

位置をずらした二配置を別々に最後まで処理し、同寸法の候補A、Bを作る。完成後に場所ごとにA、B、入力維持を選ぶ。単一格子法と同じ省メモリ性を保ちながら、格子位置への依存と不適切な描き足しを抑える。

3. 提案手法

3.1 二つの格子配置

一領域では1280×1280をモデルへ見せ、中央960×960を候補へ戻す。周囲160ピクセルは文脈として使う。次の中央部は880ピクセル先なので、隣同士は80ピクセル重なる。

出力2897×4096に対し、配置Aは4列×5行の20領域、配置Bは格子を縦横へ440ピクセルずらした4列×6行の24領域である。AだけでもBだけでも画像全体を完成させる。左右分割や市松分担ではない。端では画像外へ端画素を延長し、入力寸法を保つ。

AとBは、どちらも画像全体を処理する

左右分割ではなく、格子を半ずらした二つの完成候補を作る



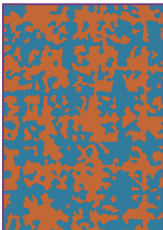
配置A：全体

20領域（4列×5行）



配置B：全体

半格子ずらした24領域



最終選択

橙=A、青=B、灰=入力、紫=弱い融合

各領域は中央960 pxを採用し、生成時は周囲160 pxを含む1280 pxを見る。隣とは80 px重なる。

図1 二つの配置と実測選択図。A、Bはいずれも画像全体を完成させる。完成後に橙=A、青=B、灰=入力、紫=弱い融合として選ぶ。

3.2 配置ごとの独立再構成

入力補間画像に対する、配置g、領域tの変化を Δ_g 、窓重みを w_g とする。配置ごとに重み付き和 D_g 、重み和 W_g 、二乗量 E_g を蓄積し、他方を混ぜず候補変化 R_g を求める。

$$D_g = \sum t \text{ wgt } \Delta g t, W_g = \sum t \text{ wgt}, E_g = \sum t \text{ wgt mean}(\Delta g t^2) \quad (1)$$

$$R_g = D_g / W_g \quad (2)$$

同じ配置内で重なる領域のばらつきから、結果がどれだけ揃うかを表す安定度 C_g を求める。ここまではAとBを混ぜない。

3.3 低周波を入力へ寄せる

候補差分を標準偏差12ピクセル相当の低周波 R^L と高周波 R^H に分ける。高周波は保ち、低周波は輝度0.32、色差0.18へ弱める。入力輝度0.85以上では輝度係数をさらに半減する。

$$R_g^L = G12(R_g), R_g^H = R_g - R_g^L \quad (3)$$

$$R^*g = R_g^H + \eta Y R_g^L, Y + \eta C R_g^L, C \quad (4)$$

3.4 細部量と入力忠実度

変化の大きさだけを品質とはみなさない。高周波量 H_g に、配置内安定度と入力忠実度 F_g を掛ける。 d_{edge} は輪郭方向の不一致、 d_{low} は低周波輝度差、 d_{chroma} は低周波色差である。

$$Q_g = H_g (0.25 + 0.75 C_g) F_g \quad (5)$$

$$F_g = \exp(-1.25 d_{edge} - 1.00 d_{low} - 0.75 d_{chroma}) \quad (6)$$

3.5 A、B、入力維持の三値選択

入力輝度を案内画像として得点を半径16、忠実度を半径8で整理し、 $S = (Q_A - Q_B) / (Q_A + Q_B + \epsilon)$ とする。 $S > 0.03$ ならB、 $S < -0.03$ ならA、それ以外は未確定とする。ただし候補の忠実度が0.42未満なら確定させない。

未確定部には周囲の確かな選択を伝える。3000画素未満のA/B島と穴を整理し、両候補が不適切なら入力補間へ戻す。二候補が十分近い場合だけ±1ピクセルで位置を合わせて弱く融合し、整理後の境界だけを5ピクセルで接続する。

3.6 補助候補を弱く使う

選択差分を R^* 、位置合わせした補助差分を R^*_{align} 、近さをaとする。局所構造類似度が0.90以上で輪郭方向も揃う場合だけ補助する。aを二乗するため、かなり近いときに限って最大10%が働く。

$$R_{sup} = R^* + 0.10 a^2 (R_{align} - R^*) \quad (7)$$

$$R_{out} = (0.90 + 0.10 a) R_{sup} \quad (8)$$

表1 提案法の主要設定

項目	値
配置A / B	20 / 24領域
モデル入力 / 採用部	1280 / 960 px
間隔 / 重なり	880 / 80 px
得点 / 忠実度の案内半径	16 / 8 px
選択しきい値 τ	0.03
忠実度の最低値	0.42
島除去 / 境界接続	3000 / 5 px
低周波係数（輝度 / 色差）	0.32 / 0.18
補助上限 / 保持下限	0.10 / 0.90

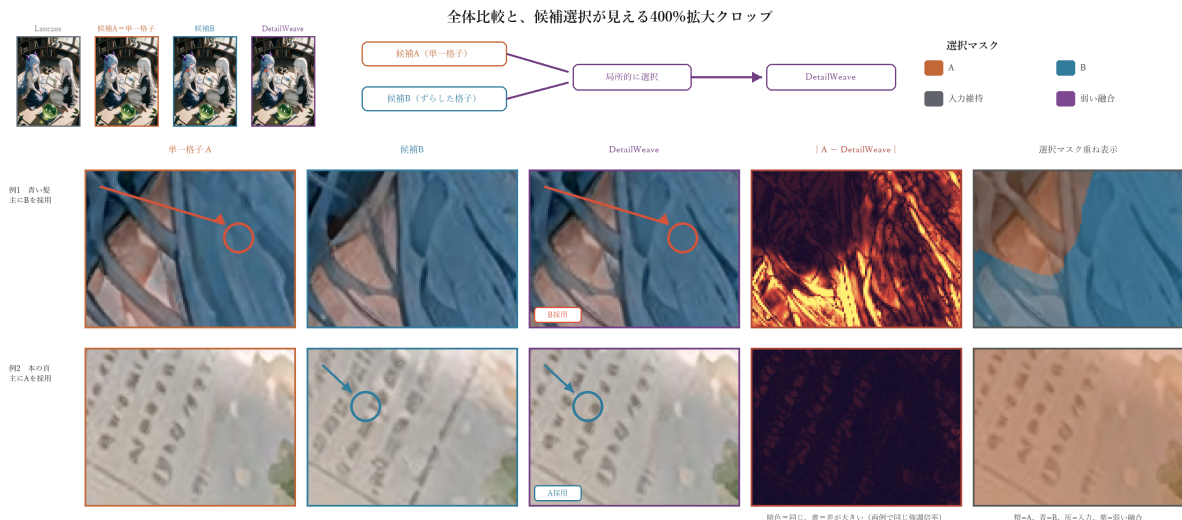


図2上：Lanczos、候補A（単一格子）、候補B、最終像の全体比較。下：同一座標の400%拡大。矢印は候補差、差分像は |A - 最終|、右端は最終像へ選択マスクを重ねたもの。髪ではB、本の頁ではAが主に採用された。

4. 事例検証

4.1 条件

同じ入力を2897×4096へ拡大した。単一格子像はDetailWeaveの配置Aを全面採用したもので、二方式は同じ局所生成を共有する。生成には独自に調整したKrea2 Turboモデル、同じ指示文、各領域6回（Exact Steps）、変化強度0.16を用いた。これは実験条件であり、選択式には関与しない。

表2 事例画像の処理条件

方法	処理
Lanczos	補間のみ
単一格子分割	配置Aの20領域を全面採用
DetailWeave	A 20領域 + B 24領域
生成条件	各領域6回（Exact Steps）、変化強度0.16

4.2 評価方法

Lanczos像を正解ではなく入力維持の基準とする。細かな明暗変化は、輝度から2ピクセル相当の平滑成分を引いた絶対値で測り、Lanczosを1とした。低周波明暗ずれは12ピクセル相当の平滑像との差、RGB平均差は0から255の画素値差である。

SSIMは輝度構造の近さを表し、1に近いほどLanczos像へ近い。境界ジャンプ比Jは、A/B両配置の計画境界における残差ジャンプの95百分位を、画像全体の同値で割る。J=1は全体と同程度、J>1は境界への集中、J<1では小さいほど境界変化が相対的に弱い。いずれも細部の正しさを直接判定しないため、図2を併用する。

5. 結果

三方式とも2897×4096を出力した。単一格子像と提案法は一回の同じ領域生成から得ており、差は二配置の完成後に行う選択・拒否・接続にある。

表3 Lanczos像を基準とした測定

評価量	Lanczos	単一格子	提案法
細かな明暗変化	1.000	1.040	1.014
低周波明暗ずれ	0.000	0.336	0.338
輝度SSIM	1.000	0.749	0.760
RGB平均差	0.000	7.724	7.508
境界ジャンプ比J↓	-	0.990	0.978

単一格子の細かな明暗変化は1.040倍、提案法は1.014倍であった。提案法は変化量を最大化せず、輝度SSIMを0.749から0.760、RGB平均差を7.724から7.508へ改善した。

境界ジャンプ比Jは単一格子の0.990から0.978へわずかに低下し、計画境界への変化集中は増えなかった。図2の髪ではB、本の頁ではAが主に選ばれ、全体でもA 49.80%、B 49.84%であった。

6. 標準的な分割処理との違い

6.1 分割VAEは入口と出口を分ける

VAEは画像を潜在画像へ変換し、最後に画像へ戻す。分割VAEはこの計算を小領域へ分けるため、VAEのGPU作業量を抑えられる。ただし一般的な実装は、全体潜在画像、全体RGB出力、加算・重みバッファをCPUメモリーに保持する。新しい細部を生成せず、ノイズ除去や指示文にも手を加えない。

DetailWeaveが分けるのは画像から画像への局所生成の全経路である。二候補を生成して忠実度を評価し、A、Bを拒否して入力維持も選べる。分割VAEの代替ではなく、その上で動く生成・選択層であり、各領域には通常VAEと分割VAEのどちらも使える。MultiDiffusion [2] はさらに別で、各段階の予測を一つの共有潜在画像へ戻す。

6.2 単一格子に加えるもの

単一格子分割とDetailWeaveは、一度に見る領域を固定するため、どちらもGPUメモリー使用量を出力解像度から切り離せる。単一格子は一つの完成像を採用する。DetailWeaveは半格子ずらした二候補を保ち、局所的にA、B、入力維持を選ぶ。

6.3 用途による使い分け

表4 目的に応じた方式の選択

用途・重視する点	向いている方式
VAE変換だけを省メモリ化	分割VAE
計算量を抑えた局所生成	単一格子
元画像に忠実な高精細拡大	DetailWeave
髪や文字など細線を守る	DetailWeave
不適切な生成を入力へ戻す	DetailWeave
GPUメモリーと出力解像度を分離	単一格子・DetailWeave
32K・64Kへ段階的に拡大	単一格子・DetailWeave

出力を2倍にすると領域数、処理時間、一時保存量はおおむね4倍になるが、一領域のGPU作業量は変わらない。この局所生成部は32K・64Kにも同じ処理を反復できる。最終復号と保存まで総メモリーを一定にする場合は、全体RGBを保持しない帯状の逐次復号・逐次保存を組み合わせる。

7. 結論

DetailWeaveは分割VAEを置き換えず、単一格子へ二候補と入力維持の選択を加える。実測では入力構造への近さを改善し、計画境界への変化集中を増やさなかった。

参考文献

- [1] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," Proc. CVPR, pp. 10684-10695, 2022.
- [2] O. Bar-Tal et al., "MultiDiffusion: Fusing Diffusion Paths for Controlled Image Generation," Proc. ICML, PMLR 202, pp. 1737-1752, 2023.
- [3] K. He, J. Sun, and X. Tang, "Guided Image Filtering," Proc. ECCV, pp. 1-14, 2010.
- [4] Z. Wang et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE Trans. Image Processing, 13(4), pp. 600-612, 2004.